What is a JOIN?

Definition

A **JOIN** is a SQL operation used to **combine rows from two or more tables** based on a common column (usually a **Primary Key** and a **Foreign Key**).

Simple Definition

> **JOIN allows us to retrieve related data stored in different tables.**

Why Do We Need JOINs?

Imagine a college database.

Instead of storing everything in one table:

Student_ID	Student_Name	Course_Name	Teacher
101	Rahul	Java	Amit
102	Priya	Python	Ravi

This causes **data duplication**.

Instead, databases store data in separate tables.

Students Table

Student_ID	Student_Name
101	Rahul
102	Priya
103	Aman
104	Sneha

Courses Table

Course_ID	Course_Name
1	Java
2	Python
3	SQL

Enrollment Table

Student_ID	Course_ID
101	1
102	2
103	1

Now information is stored efficiently.

JOIN combines these tables whenever required.

Understanding Primary Key & Foreign Key

Primary Key

A **Primary Key** uniquely identifies every row in a table.

Example

```
Students

+------------+---------+
| Student_ID | Name    |
+------------+---------+
| 101        | Rahul   |
| 102        | Priya   |
| 103        | Aman    |
+------------+---------+
```

Student_ID is the Primary Key.

Properties

- Unique
- Cannot be NULL
- One Primary Key per table

Foreign Key

A **Foreign Key** is a column that refers to the Primary Key of another table.

```
Enrollment

+------------+-----------+
| Student_ID | Course_ID |
+------------+-----------+
| 101        | 1         |
| 102        | 2         |
| 103        | 1         |
+------------+-----------+
```

Student_ID is a Foreign Key.

Database Used in Examples

Students

```
CREATE TABLE Students(
Student_ID INT PRIMARY KEY,
Name VARCHAR(30)
);
```

Data

Student_ID	Name
101	Rahul
102	Priya
103	Aman
104	Sneha

Courses

```
CREATE TABLE Courses(
Course_ID INT PRIMARY KEY,
Course_Name VARCHAR(30)
);
```

Data

Course_ID	Course_Name
1	Java
2	Python
3	SQL

Enrollment

```
CREATE TABLE Enrollment(
Student_ID INT,
Course_ID INT
);
```

Data

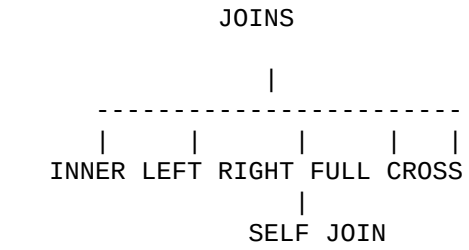
Student_ID	Course_ID
101	1
102	2
103	1

Notice

Sneha is **not enrolled**.

SQL course has **no student**.

Types of JOINs



---

1. INNER JOIN

Definition

Returns **only matching rows** from both tables.

If there is no match, the row is ignored.

Diagram

```
Students                      Enrollment

101 Rahul  ------------\
                        \
                         ---> Match ---> Result
                        /
102 Priya ------------/

104 Sneha (No Match)

Result

Rahul
Priya
Aman
```

Only common data appears.

SQL Query

```
SELECT
Students.Name,
Courses.Course_Name
FROM Students
INNER JOIN Enrollment
ON Students.Student_ID = Enrollment.Student_ID
INNER JOIN Courses
ON Enrollment.Course_ID = Courses.Course_ID;
```

Output

Name	Course
Rahul	Java
Priya	Python
Aman	Java

Sneha is missing because she has no matching enrollment.

Venn Diagram





**** = Returned

Only the overlapping portion is returned.

Real-Life Example

Imagine two lists.

Students

```
Rahul
Priya
Aman
Sneha
```

Library Members

```
Rahul
Priya
Aman
```

INNER JOIN returns

```
Rahul
Priya
Aman
```

---

2. LEFT JOIN

Definition

Returns

✓ All rows from the LEFT table

-

✓ Matching rows from the RIGHT table

If no match exists

NULL is returned.

Diagram

```
LEFT TABLE

Rahul
Priya
Aman
Sneha

RIGHT TABLE
```

```
Java
Python
Java

Output

Rahul -> Java
Priya -> Python
Aman -> Java
Sneha -> NULL
```

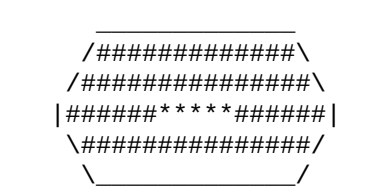
SQL

```
SELECT
Students.Name,
Courses.Course_Name
FROM Students
LEFT JOIN Enrollment
ON Students.Student_ID = Enrollment.Student_ID
LEFT JOIN Courses
ON Enrollment.Course_ID = Courses.Course_ID;
```

Output

Name	Course
Rahul	Java
Priya	Python
Aman	Java
Sneha	NULL

Venn Diagram



```
# = Returned
* = Matching
```

Entire left circle is returned.

Real-Life Example

Class Students

```
Rahul
Priya
Aman
Sneha
```

Fee Paid

```
Rahul
Priya
Aman
```

LEFT JOIN shows

```
Rahul Paid
Priya Paid
Aman Paid
Sneha NULL
```

---

3. RIGHT JOIN

Definition

Returns every row from the RIGHT table.

Matching rows from LEFT are shown.

Otherwise NULL.

Diagram

```
RIGHT TABLE

Java
Python
SQL

LEFT TABLE

Rahul
Priya
Aman

Output

Rahul Java
Aman Java
Priya Python
NULL SQL
```

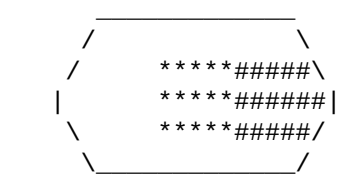
SQL

```
SELECT
Students.Name,
Courses.Course_Name
FROM Students
RIGHT JOIN Enrollment
ON Students.Student_ID = Enrollment.Student_ID
RIGHT JOIN Courses
ON Enrollment.Course_ID = Courses.Course_ID;
```

Output

Name	Course
Rahul	Java
Aman	Java
Priya	Python
NULL	SQL

Venn Diagram



```
# = Returned
```

Entire right circle is returned.

Real-Life Example

Courses

```
Java
Python
SQL
```

Students

```
Rahul Java
Priya Python
Aman Java
```

RIGHT JOIN shows

```
Java
Python
SQL
```

Even SQL has no student.

---

4. FULL OUTER JOIN

Definition

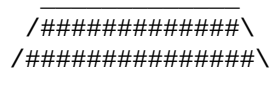
Returns

- All matching rows
- All rows from left table
- All rows from right table

MySQL does **not** support `FULL OUTER JOIN` directly.

Use `LEFT JOIN` + `UNION` + `RIGHT JOIN`.

Diagram



```
    |######*****######|
     \###############/
      \_____________/
```

Everything is returned.

SQL

```
SELECT
Students.Name,
Courses.Course_Name
FROM Students
LEFT JOIN Enrollment
ON Students.Student_ID = Enrollment.Student_ID
LEFT JOIN Courses
ON Enrollment.Course_ID = Courses.Course_ID

UNION

SELECT
Students.Name,
Courses.Course_Name
FROM Students
RIGHT JOIN Enrollment
ON Students.Student_ID = Enrollment.Student_ID
RIGHT JOIN Courses
ON Enrollment.Course_ID = Courses.Course_ID;
```

Output

Name	Course
Rahul	Java
Priya	Python
Aman	Java
Sneha	NULL
NULL	SQL

Everything appears.

---

5. CROSS JOIN

Definition

Returns **every possible combination** of rows.

No matching condition is required.

Formula

```
Total Rows

=

Rows in First Table

×
```

```
Rows in Second Table
```

Students

```
Rahul
Priya
```

Courses

```
Java
Python
SQL
```

Diagram

```
Rahul

   |----Java
   |----Python
   |----SQL

Priya

   |----Java
   |----Python
   |----SQL
```

SQL

```
SELECT
Students.Name,
Courses.Course_Name
FROM Students
CROSS JOIN Courses;
```

Output

Student	Course
Rahul	Java
Rahul	Python
Rahul	SQL
Priya	Java
Priya	Python
Priya	SQL

2 × 3 = 6 rows

---

6. SELF JOIN

Definition

A **SELF JOIN** joins a table with itself.

Used when rows within the same table are related.

Employees Table

EmpID	Employee	ManagerID
1	John	NULL
2	Rahul	1
3	Priya	1

Diagram

```
Employees Table

John
 ↑
 |
Rahul

John
 ↑
 |
Priya
```

SQL

```
SELECT
E.Employee,
M.Employee AS Manager
FROM Employees E
LEFT JOIN Employees M
ON E.ManagerID = M.EmpID;
```

Output

Employee	Manager
John	NULL
Rahul	John
Priya	John

Summary Table

JOIN	Returns
INNER JOIN	Only matching rows
LEFT JOIN	All left rows + matching right rows
RIGHT JOIN	All right rows + matching left rows
FULL OUTER JOIN	All rows from both tables (simulated with `LEFT JOIN` + `UNION` + `RIGHT JOIN` in MySQL)
CROSS JOIN	Every possible combination
SELF JOIN	A table joined with itself

Memory Trick

```
INNER JOIN
→ Intersection only

LEFT JOIN
→ Everything on LEFT + Match

RIGHT JOIN
→ Everything on RIGHT + Match

FULL OUTER JOIN
→ Everything from BOTH

CROSS JOIN
→ Every Combination

SELF JOIN
→ Same Table with Itself
```

Practice Questions

1. What is a JOIN in MySQL?
2. Why are JOINs used?
3. Explain the difference between a Primary Key and a Foreign Key.
4. What is the difference between `INNER JOIN` and `LEFT JOIN`?
5. What happens when there is no matching record in a `LEFT JOIN`?
6. Why doesn't MySQL support `FULL OUTER JOIN` directly?
7. When would you use a `CROSS JOIN`?
8. Explain a `SELF JOIN` with an employee-manager example.
9. Can you join more than two tables? If yes, how?
10. Write SQL queries for each type of JOIN using the `Students`, `Courses`, and `Enrollment` tables.